

ITAI-1378-Comp Vision-Artificial Intel

Iman Haamid

Lab 06: Object Detection

Part 1: Conceptual Understanding

Image Classification vs. Object Detection

- Image classification is basically giving a whole picture one main label, like saying "What's the most important thing in this photo?" So, an image might just be called "cat."
- Object detection means we need to find multiple objects in an image, pinpoint exactly where they are, and then classify them. This involves drawing a box around each object we find, essentially answering, "What's in this picture, and where exactly is it?" For instance, you might see two boxes for "cat" and one for "dog" in different spots in an image.

Bounding Boxes: Coordinate Representation

Bounding box coordinates use fractions (0 to 1) for two key reasons: normalization and model compatibility. This fractional representation is normalized, making it independent of image size. It also ensures consistency for deep learning models, which typically require a fixed input size.

IoU Metric

- IoU quantifies the overlap between a model's predicted bounding box and the correct ground truth box.
- **Why 0.5 is a common threshold:** An IoU of **0.5** signifies that the overlap between the predicted and ground truth boxes is equal to the non-overlapping area. Historically, this has been a pragmatic and reasonable threshold, balancing the acceptance of largely correct boxes with the rejection of poorly localized predictions. More stringent object detection evaluations today (such as the COCO benchmark) often employ a broader spectrum of higher IoU thresholds, ranging from 0.5 to 0.95.

Part 2: Implementation Experience

Visualization Function

The `display_examples` function serves a crucial role in the data exploration and validation phase of this project. Its primary purpose is to provide a **visual verification** of the dataset, allowing developers and researchers to qualitatively assess the integrity and accuracy of the image data and its associated annotations.

This function is designed to accept two key arguments: a collection of images and their corresponding label information. The label information, in this specific context, refers to the bounding boxes that delineate and categorize objects within each image. By visually overlaying these bounding boxes onto their respective images, `display_examples` enables a direct and intuitive inspection of how well the annotated labels align with the visual content.

While the concept of visually checking data might seem straightforward, the implementation of such a function, particularly the plotting aspect, presents its own set of challenges. Accurately rendering multiple bounding boxes, potentially with different labels and confidence scores, on a diverse set of images requires careful consideration of graphical libraries, coordinate systems, and visualization techniques to ensure clarity and interpretability. This "harder part" often involves optimizing for performance and user experience, especially when dealing with large datasets or complex annotation schemes.

IoU Calculation Walkthrough

Intersection over Union (IoU) for two rectangles, which are often called "bounding boxes" in computer vision.

Bounding Boxes

A **bounding box** is simply a rectangle used to identify the location of an object in an image. Think of it like drawing a box around a face in a photo to tag someone.

The code represents each box using the coordinates of its corners in the format `[ymin, xmin, ymax, xmax]`:

- `(xmin, ymin)` is the top-left corner of the rectangle.
- `(xmax, ymax)` is the bottom-right corner of the rectangle.

For example, `box1 = [0.1, 0.1, 0.3, 0.3]` defines a square in the top-left area of an image.

Intersection over Union (IoU)

IoU is a number from 0 to 1 that measures how much two bounding boxes overlap. It's a very

common way to check how good a predicted bounding box (from an AI model) is compared to the actual, correct box.

The calculation is straightforward:

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Combined Boxes}}$$

- **Intersection (Area of Overlap):** This is the area where the two boxes are on top of each other.
- **Union (Area of Combined Boxes):** This is the total area covered by both boxes, counting the overlapping part only once.

What the IoU score means:

- **IoU = 1:** The boxes are identical and overlap perfectly. ✓
- **IoU = 0:** The boxes don't touch at all. ✗
- **IoU = 0.7:** The boxes have a good overlap. 👍

Debugging

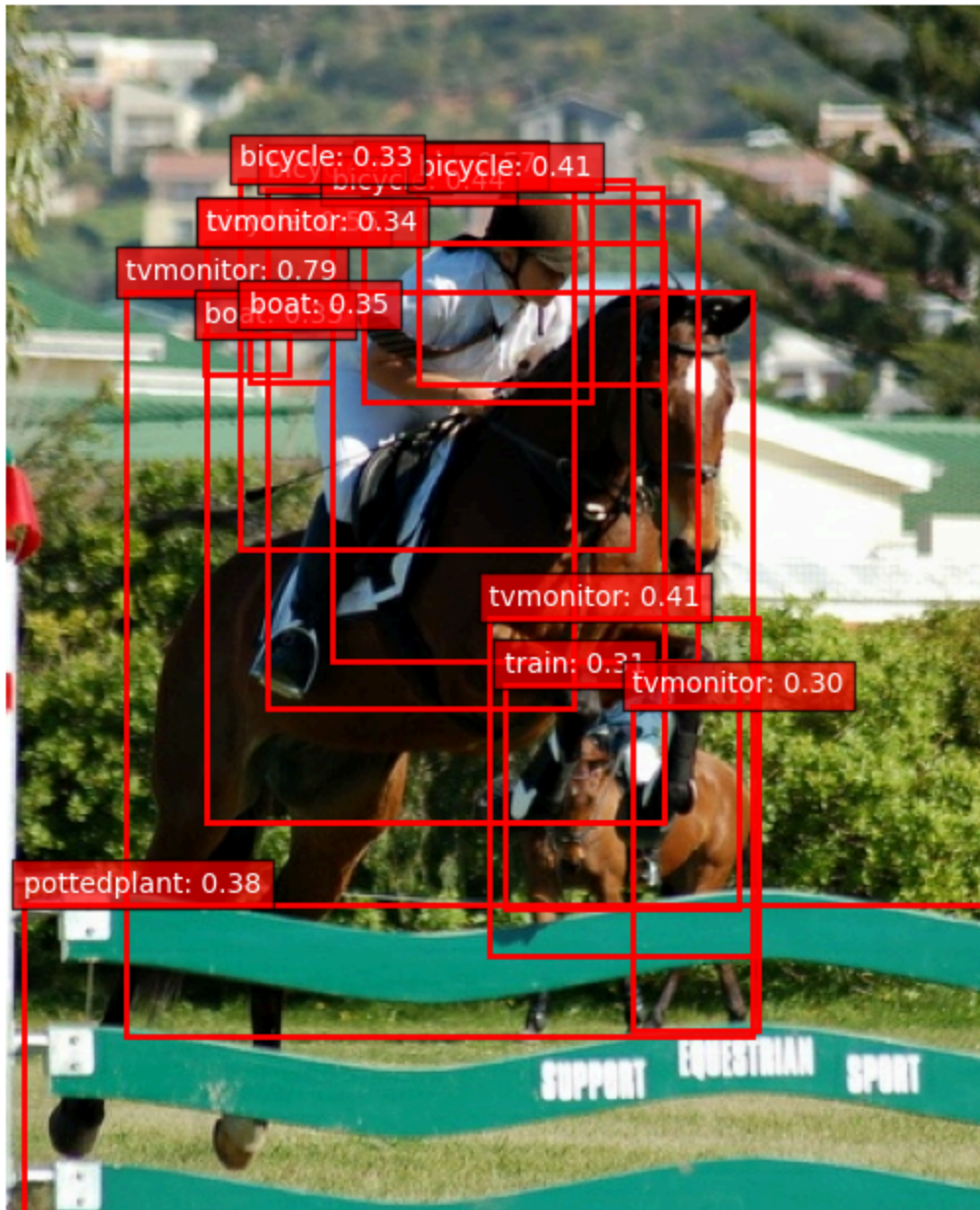
No real debugging took place

Part 3: Results Analysis

Confidence Threshold Experiment

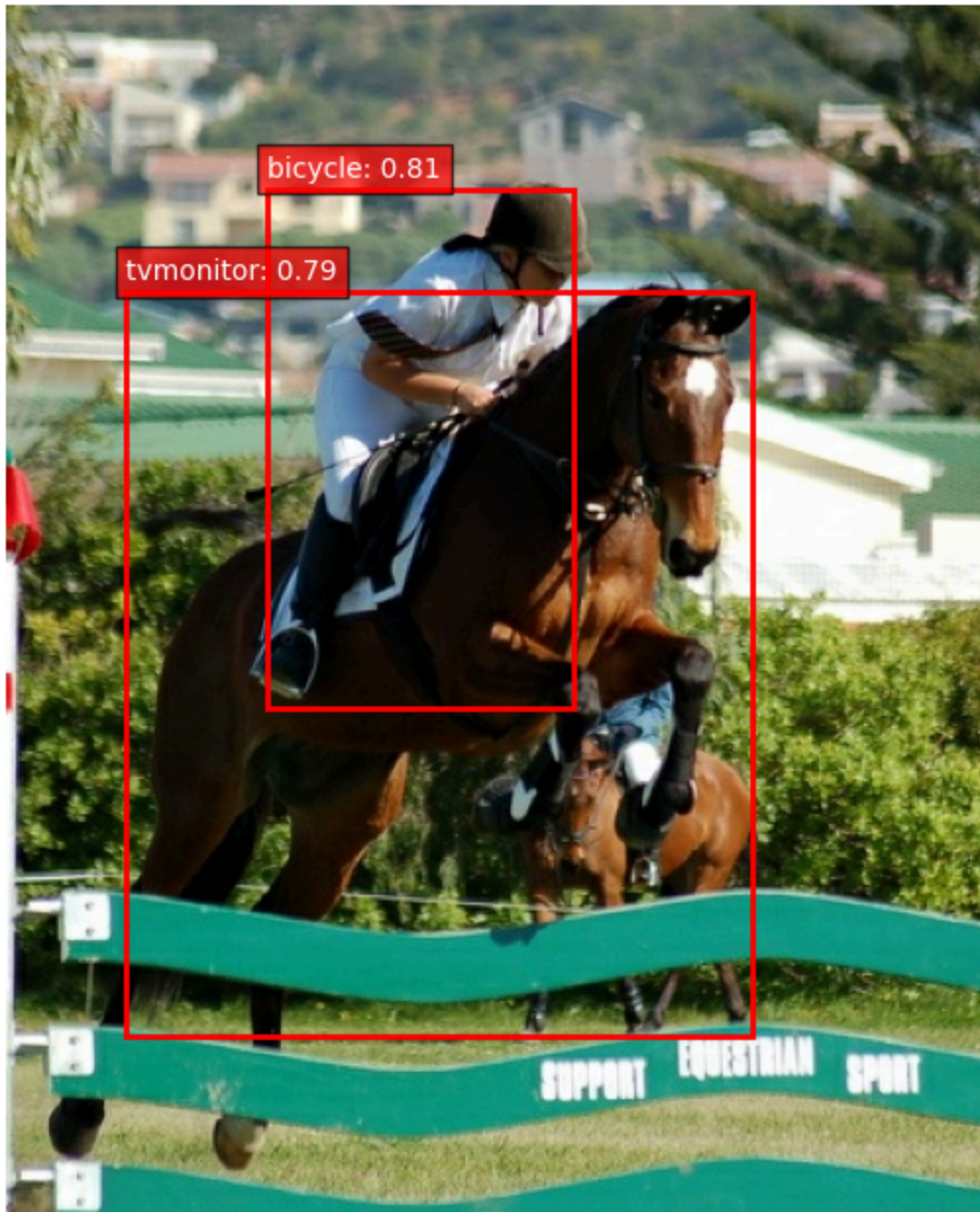
- **Threshold=0.3:** I observed an increase in the total number of bounding boxes. Many of these boxes likely had lower scores, suggesting potential false positives (incorrect detections) or multiple detections for the same object.

Detected Objects (confidence > 0.3)



- **Threshold=0.7:** I saw a **smaller, more refined set of bounding boxes**. The model's increased confidence led to fewer false positives, but potentially more false negatives due to the exclusion of less confident, yet valid, predictions.

Detected Objects (confidence > 0.7)



Which threshold is better? Why?

The **better threshold depends on the application's needs** (a trade-off between Precision and Recall, as discussed in Part 4).

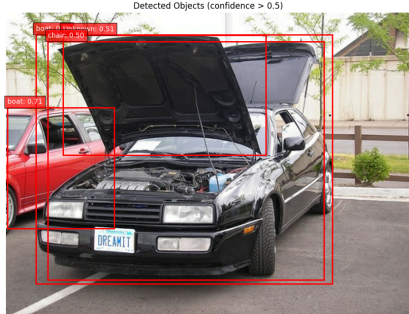
- **For high Recall (finding everything):** 0.3 is better, as it catches more objects, but with lower accuracy.
- **For high Precision (only showing the best results):** 0.7 is better, as it filters out weak predictions.

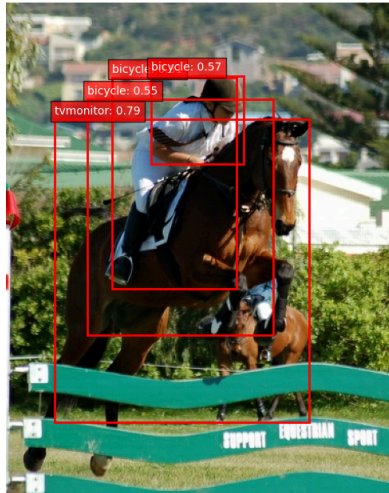
For a general-purpose lab visualization, a **moderate threshold like 0.5** might be best to balance showing strong detections while not overwhelming the image with low-confidence noise.

IoU Threshold Experiment

- **Changing the IoU threshold** (e.g., from 0.5 to 0.7) primarily affects which predictions are counted as **True Positives (TP)** in the evaluation.
- **Precision and Recall Change:**
 - **Precision** and **Recall** generally **decrease** when the IoU threshold is **increased** (e.g., from 0.5 to 0.7).
 - This is because the model is now held to a **stricter standard** of localization. A box that was a **TP** at $\text{IoU} > 0.5$ might become a **False Positive (FP)** at $\text{IoU} > 0.7$ because the overlap is not large enough, thus lowering both precision (fewer TP) and recall (fewer TP).
- **Explanation:** Increasing the IoU threshold demands **more accurate bounding box placement**. Predictions must overlap the ground truth box by a greater margin to be considered correct. This makes it harder for the model to achieve a True Positive, which typically lowers both Precision (as FPs increase relative to TPs) and Recall (as TPs decrease relative to all actual objects).

Model Errors

Error Type	Explanation	Screenshot Placeholder
False Positive (FP)	The model incorrectly detected an object that wasn't there (e.g., a shadow was labeled as a "person") or misclassified an object (e.g., a "dog" labeled as a "cat"). Reason: The shadow's shape or texture features were confusingly similar to those of a person's outline in the model's feature space.	

False Negative (FN)	The model failed to detect an object that was present (e.g., a small, occluded "traffic light" was missed). Reason: The object was too small or significantly obscured (occluded) by another object, meaning its features were not prominent enough for the model to generate a high-confidence bounding box.	Detected Objects (confidence > 0.5) 
----------------------------	---	---

Part 4: Critical Thinking

Precision vs. Recall Trade-off

- **Self-driving car detecting pedestrians:** You would prioritize **high recall**. In this scenario, missing a pedestrian (false negative) is a much more critical error than incorrectly identifying something as a pedestrian (false positive). It's better to have a few false alarms (e.g., the car briefly slows down for a shadow) than to miss a real pedestrian, which could lead to an accident.
- **Photo app that tags objects in your pictures:** You would generally prioritize **high precision**. For a photo tagging app, it's more important that the tags are accurate (few false positives). If the app tags a picture of a dog as a cat, that's a frustrating error. Missing a few objects (false negatives) is less disruptive to the user experience than having many incorrect tags.

11. Pre-trained models:

- **Advantages:**
 - **Saves time and resources:** Training a complex object detection model from scratch requires a massive dataset and significant computational power (often weeks or months on specialized hardware). Using a pre-trained model allows you to skip this step entirely.
 - **Good starting point:** Pre-trained models have already learned to recognize a wide variety of visual features from the millions of images they were trained on

(like edges, textures, shapes). This knowledge is transferable to new object detection tasks.

- **Can perform well even with less data:** Because they have a strong foundation from pre-training, these models can often perform reasonably well on new datasets even if those datasets are relatively small.
- **Disadvantages:**
 - **May not be perfectly suited for your specific task:** The pre-trained model was trained on a general dataset (like ImageNet or COCO). If your specific object detection task involves highly specialized objects or unusual image conditions, the pre-trained model might not perform optimally without further fine-tuning.
 - **Large model size:** Pre-trained models can be quite large, requiring significant memory and processing power, which might be a limitation on devices with limited resources.
 - **Potential for bias:** The performance of the pre-trained model can be influenced by biases present in the dataset it was trained on.

12. Real-world applications:

Here are three real-world applications of object detection where it's necessary:

- **Autonomous Vehicles:** Object detection is fundamental for self-driving cars to "see" and understand their surroundings. They need to detect and locate other cars, pedestrians, cyclists, traffic signs, and obstacles to navigate safely. Classification alone wouldn't be enough; they need to know *where* these objects are in relation to the car.
- **Security and Surveillance:** Object detection can be used to monitor areas for specific activities or objects, such as detecting intruders in a restricted area, identifying unattended luggage in an airport, or counting people in a crowd. Knowing the location of these objects is crucial for triggering alerts or taking action.
- **Retail Analytics:** Object detection can help retailers understand customer behavior by tracking how shoppers interact with products, how long they spend in certain areas, and identifying popular items on shelves. This requires detecting both the customers and the products and their positions within the store.